

Universidade do Extremo Sul Catarinense

Emanoel Clezar Baltazar
Liliane Antunes de Souza
Ricardo Moisés Nazário da Silva
Stefany França

Gerenciamento de Dados II
Plataforma de gerenciamento de Backup – Aquarismo
2º Entrega

Criciúma
2026

1 SCRIPT DE CRIAÇÃO E CARGA

1.1 SCHEMA BANCO AQUARISMO

```
CREATE TABLE clientes (  
  id      SERIAL PRIMARY KEY,  
  nome    VARCHAR(150) NOT NULL,  
  email   VARCHAR(150) UNIQUE NOT NULL,  
  telefone VARCHAR(30),  
  criado_em  TIMESTAMP NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE especies (  
  id          SERIAL PRIMARY KEY,  
  nome_popular  VARCHAR(100) NOT NULL,  
  nome_cientifico VARCHAR(150),  
  categoria    VARCHAR(20) NOT NULL CHECK (categoria IN  
( 'peixe', 'planta', 'invertebrado' )),  
  nivel_dificuldade VARCHAR(20) CHECK (nivel_dificuldade IN  
( 'facil', 'medio', 'dificil' ))  
);
```

```
CREATE TABLE lotes (  
  id          SERIAL PRIMARY KEY,  
  especie_id  INTEGER NOT NULL REFERENCES especies(id),  
  fornecedor  VARCHAR(150),  
  data_chegada  DATE NOT NULL DEFAULT CURRENT_DATE,  
  quantidade_inicial INTEGER NOT NULL CHECK (quantidade_inicial >=  
0),  
  quantidade_atual  INTEGER NOT NULL CHECK (quantidade_atual >=  
0),  
  preco_unitario  NUMERIC(10,2) NOT NULL CHECK (preco_unitario >=  
0)  
);
```

```
CREATE TABLE produtos_acessorios (  
  id          SERIAL PRIMARY KEY,  
  nome        VARCHAR(150) NOT NULL,  
  categoria   VARCHAR(20) NOT NULL CHECK (categoria IN  
( 'racaio', 'filtro', 'decoracao', 'equipamento' )),  
  preco       NUMERIC(10,2) NOT NULL CHECK (preco >= 0),  
  estoque_atual  INTEGER NOT NULL DEFAULT 0 CHECK  
(estoque_atual >= 0)  
);
```

```
CREATE TABLE pedidos (  
  id          SERIAL PRIMARY KEY,
```

```

    cliente_id INTEGER NOT NULL REFERENCES clientes(id),
    data_pedido TIMESTAMP NOT NULL DEFAULT now(),
    status VARCHAR(20) NOT NULL DEFAULT 'pendente' CHECK
(status IN ('pendente','pago','enviado','cancelado')),
    total NUMERIC(12,2) NOT NULL DEFAULT 0
);

```

```

CREATE TABLE itens_pedido (
    id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedidos(id) ON
DELETE CASCADE,
    tipo_item VARCHAR(10) NOT NULL CHECK (tipo_item IN
('peixe','acessorio')),
    lote_id INTEGER REFERENCES lotes(id),
    produto_id INTEGER REFERENCES produtos_acessorios(id),
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),
    preco_unitario NUMERIC(10,2) NOT NULL,
    CONSTRAINT chk_item_referencia CHECK (
        (tipo_item = 'peixe' AND lote_id IS NOT NULL AND produto_id IS
NULL) OR
        (tipo_item = 'acessorio' AND produto_id IS NOT NULL AND lote_id IS
NULL)
    )
);

```

```

CREATE TABLE movimentacoes_estoque (
    id SERIAL PRIMARY KEY,
    lote_id INTEGER REFERENCES lotes(id),
    produto_id INTEGER REFERENCES produtos_acessorios(id),
    tipo VARCHAR(10) NOT NULL CHECK (tipo IN
('entrada','venda','perda')),
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),
    data TIMESTAMP NOT NULL DEFAULT now(),
    observacao VARCHAR(255),
    CONSTRAINT chk_mov_referencia CHECK (
        (lote_id IS NOT NULL AND produto_id IS NULL) OR
        (lote_id IS NULL AND produto_id IS NOT NULL)
    )
);

```

```

CREATE INDEX idx_lotes_especie ON lotes(especie_id);
CREATE INDEX idx_pedidos_cliente ON pedidos(cliente_id);
CREATE INDEX idx_itens_pedido_pedido ON itens_pedido(pedido_id);
CREATE INDEX idx_mov_estoque_data ON
movimentacoes_estoque(data);
CREATE INDEX idx_mov_estoque_lote ON
movimentacoes_estoque(lote_id);

```

1.2 SCHEMA BANCO BACKUP

```
CREATE TABLE configuracoes_backup (  
  id          SERIAL PRIMARY KEY,  
  banco       VARCHAR(100) NOT NULL,  
  caminho_destino VARCHAR(255) NOT NULL,  
  qtd_manter  INTEGER NOT NULL DEFAULT 7,  
  caminho_copia  VARCHAR(255),  
  criptografar  BOOLEAN NOT NULL DEFAULT false,  
  compactar     BOOLEAN NOT NULL DEFAULT false,  
  criado_em     TIMESTAMP NOT NULL DEFAULT now()  
);  
  
CREATE TABLE execucoes (  
  id          SERIAL PRIMARY KEY,  
  config_id   INTEGER NOT NULL REFERENCES  
configuracoes_backup(id),  
  data_inicio  TIMESTAMP NOT NULL DEFAULT now(),  
  data_fim     TIMESTAMP,  
  decisao_manutencao VARCHAR(30) CHECK (decisao_manutencao  
IN  
( 'NENHUMA', 'VACUUM', 'VACUUM_ANALYZE', 'VACUUM_FULL_ANALYZE'  
)),  
  regra_aplicada  VARCHAR(255),  
  status         VARCHAR(20) NOT NULL DEFAULT 'em_andamento'  
CHECK (status IN ('em_andamento', 'sucesso', 'falha')),  
  resultado      VARCHAR(255)  
);  
  
CREATE TABLE logs_execucao (  
  id          SERIAL PRIMARY KEY,  
  execucao_id INTEGER NOT NULL REFERENCES execucoes(id) ON  
DELETE CASCADE,  
  etapa      VARCHAR(100) NOT NULL,  
  mensagem   VARCHAR(500) NOT NULL,  
  data       TIMESTAMP NOT NULL DEFAULT now(),  
  saida_tecnica TEXT  
);  
  
CREATE INDEX idx_execucoes_config ON execucoes(config_id);  
CREATE INDEX idx_execucoes_data ON execucoes(data_inicio);  
CREATE INDEX idx_logs_execucao ON logs_execucao(execucao_id);
```

1.3 SCRIPT DE POPULAÇÃO

```
TRUNCATE TABLE
```

```
logs_execucao,  
execucoes,  
configuracoes_backup,  
movimentacoes_estoque,  
itens_pedido,  
pedidos,  
lotes,  
produtos_acessorios,  
especies,  
clientes
```

```
RESTART IDENTITY CASCADE;
```

```
INSERT INTO especies (
```

```
nome_popular,  
nome_cientifico,  
categoria,  
nivel_dificuldade
```

```
)
```

```
VALUES
```

```
('Betta', 'Betta splendens', 'peixe', 'facil'),  
( 'Neon Tetra', 'Paracheirodon innesi', 'peixe', 'facil'),  
( 'Peixe-palhaço', 'Amphiprion ocellaris', 'peixe', 'medio'),  
( 'Guppy', 'Poecilia reticulata', 'peixe', 'facil'),  
( 'Acará-bandeira', 'Pterophyllum scalare', 'peixe', 'medio'),  
( 'Molinésia', 'Poecilia sphenops', 'peixe', 'facil'),  
( 'Peixe-dourado', 'Carassius auratus', 'peixe', 'facil'),  
( 'Camarão Neocaridina', 'Neocaridina davidi', 'invertebrado', 'medio'),  
( 'Caramujo Neritina', 'Neritina natalensis', 'invertebrado', 'facil'),  
( 'Anúbia', 'Anubias barteri', 'planta', 'facil'),  
( 'Musgo de Java', 'Taxiphyllum barbieri', 'planta', 'facil'),  
( 'Cabomba', 'Cabomba caroliniana', 'planta', 'medio'),  
( 'Elódea', 'Egeria densa', 'planta', 'facil'),  
( 'Amazonense', 'Echinodorus amazonicus', 'planta', 'medio'),  
( 'Rabo de Raposa', 'Ceratophyllum demersum', 'planta', 'facil');
```

```
INSERT INTO clientes (
```

```
nome,  
email,  
telefone
```

```
)
```

```
SELECT
```

```
CASE (g % 12)  
  WHEN 0 THEN 'Ana'  
  WHEN 1 THEN 'Bruno'
```

```

        WHEN 2 THEN 'Camila'
        WHEN 3 THEN 'Daniel'
        WHEN 4 THEN 'Eduarda'
        WHEN 5 THEN 'Felipe'
        WHEN 6 THEN 'Gabriela'
        WHEN 7 THEN 'Henrique'
        WHEN 8 THEN 'Isabela'
        WHEN 9 THEN 'João'
        WHEN 10 THEN 'Larissa'
        ELSE 'Marcos'
    END
    || ' ' ||
CASE (g % 10)
    WHEN 0 THEN 'Silva'
    WHEN 1 THEN 'Santos'
    WHEN 2 THEN 'Oliveira'
    WHEN 3 THEN 'Souza'
    WHEN 4 THEN 'Pereira'
    WHEN 5 THEN 'Costa'
    WHEN 6 THEN 'Rodrigues'
    WHEN 7 THEN 'Almeida'
    WHEN 8 THEN 'Ferreira'
    ELSE 'Martins'
END
    || ' ' || g,

'cliente' || g || '@aquarismo.test',

'(48) 9' ||
LPAD((10000000 + g)::TEXT, 8, '0')

FROM generate_series(1, 30000) AS g;

INSERT INTO produtos_acessorios (
    nome,
    categoria,
    preco,
    estoque_atual
)
SELECT
    CASE (g % 4)
        WHEN 0 THEN 'Ração Premium para Peixes'
        WHEN 1 THEN 'Filtro Externo para Aquário'
        WHEN 2 THEN 'Decoração Artificial'
        ELSE 'Equipamento de Iluminação'
    END
    || ' Modelo ' || g,

```

```

CASE (g % 4)
  WHEN 0 THEN 'raca0'
  WHEN 1 THEN 'filtro'
  WHEN 2 THEN 'decoracao'
  ELSE 'equipamento'
END,

ROUND((20 + (g % 300) * 1.75)::NUMERIC, 2),

1000

FROM generate_series(1, 2000) AS g;

INSERT INTO lotes (
  especie_id,
  fornecedor,
  data_chegada,
  quantidade_inicial,
  quantidade_atual,
  preco_unitario
)
SELECT
  ((g - 1) % 15) + 1,

  CASE (g % 6)
    WHEN 0 THEN 'Fornecedor Aquatic Life'
    WHEN 1 THEN 'Distribuidora Tropical'
    WHEN 2 THEN 'Mundo dos Aquários'
    WHEN 3 THEN 'Oceanic Brasil'
    WHEN 4 THEN 'Vida Aquática'
    ELSE 'Importadora Marinha'
  END,

  CURRENT_DATE - ((g % 720)::INTEGER),

  1000,

  1000,

  ROUND((5 + (g % 200) * 0.85)::NUMERIC, 2)

FROM generate_series(1, 10000) AS g;

```

```

INSERT INTO pedidos (
    cliente_id,
    data_pedido,
    status,
    total
)
SELECT
    ((g - 1) % 30000) + 1,

    TIMESTAMP '2025-01-01'
      + ((g % 630) * INTERVAL '1 day')
      + ((g % 86400) * INTERVAL '1 second'),

    CASE (g % 10)
      WHEN 0 THEN 'cancelado'
      WHEN 1 THEN 'pendente'
      WHEN 2 THEN 'pago'
      ELSE 'enviado'
    END,

    0

```

```

FROM generate_series(1, 70000) AS g;

```

```

INSERT INTO itens_pedido (
    pedido_id,
    tipo_item,
    lote_id,
    produto_id,
    quantidade,
    preco_unitario
)
SELECT
    ((g - 1) % 70000) + 1,

    CASE
      WHEN g % 2 = 0 THEN 'peixe'
      ELSE 'acessorio'
    END,

    CASE
      WHEN g % 2 = 0
      THEN ((g - 1) % 10000) + 1
      ELSE NULL
    END,

```

```

CASE
  WHEN g % 2 = 0
  THEN NULL
  ELSE ((g - 1) % 2000) + 1
END,

```

```

1 + (g % 5),

```

```

CASE
  WHEN g % 2 = 0
  THEN (
    SELECT preco_unitario
    FROM lotes
    WHERE id = ((g - 1) % 10000) + 1
  )
  ELSE (
    SELECT preco
    FROM produtos_acessorios
    WHERE id = ((g - 1) % 2000) + 1
  )
END

```

```

FROM generate_series(1, 140000) AS g;

```

```

UPDATE pedidos p
SET total = COALESCE(
  (
    SELECT SUM(i.quantidade * i.preco_unitario)
    FROM itens_pedido i
    WHERE i.pedido_id = p.id
  ),
  0
);

```

```

INSERT INTO movimentacoes_estoque (
  lote_id,
  produto_id,
  tipo,
  quantidade,
  data,
  observacao
)
SELECT
  ((g - 1) % 10000) + 1,

  NULL,

```

```

CASE
  WHEN g % 10 IN (0, 1) THEN 'venda'
  WHEN g % 10 IN (2, 3) THEN 'perda'
  ELSE 'entrada'
END,

1 + (g % 5),

TIMESTAMP '2025-01-01'
  + ((g % 630) * INTERVAL '1 day')
  + ((g % 86400) * INTERVAL '1 second'),

CASE
  WHEN g % 10 IN (0, 1)
    THEN 'Venda de animais aquáticos'
  WHEN g % 10 IN (2, 3)
    THEN 'Perda registrada no estoque'
  ELSE 'Entrada de novo lote'
END

FROM generate_series(1, 123992) AS g;

INSERT INTO movimentacoes_estoque (
  lote_id,
  produto_id,
  tipo,
  quantidade,
  data,
  observacao
)
SELECT
  NULL,

  ((g - 1) % 2000) + 1,

CASE
  WHEN g % 10 IN (0, 1) THEN 'venda'
  WHEN g % 10 IN (2, 3) THEN 'perda'
  ELSE 'entrada'
END,

1 + (g % 5),

TIMESTAMP '2025-01-01'
  + ((g % 630) * INTERVAL '1 day')
  + ((g % 86400) * INTERVAL '1 second'),

```

```

CASE
  WHEN g % 10 IN (0, 1)
    THEN 'Venda de acessório'
  WHEN g % 10 IN (2, 3)
    THEN 'Perda registrada no estoque'
  ELSE 'Entrada de produtos'
END

```

```

FROM generate_series(1, 123992) AS g;

```

```

UPDATE lotes l
SET quantidade_atual = GREATEST(
  0,
  l.quantidade_inicial
  +
  COALESCE(
    (
      SELECT SUM(
        CASE
          WHEN m.tipo = 'entrada' THEN m.quantidade
          WHEN m.tipo IN ('venda', 'perda')
            THEN -m.quantidade
          ELSE 0
        END
      )
      FROM movimentacoes_estoque m
      WHERE m.lote_id = l.id
    ),
    0
  )
);

```

```

UPDATE produtos_acessorios p
SET estoque_atual = GREATEST(
  0,
  1000
  +
  COALESCE(
    (
      SELECT SUM(
        CASE
          WHEN m.tipo = 'entrada' THEN m.quantidade
          WHEN m.tipo IN ('venda', 'perda')
            THEN -m.quantidade
          ELSE 0
        END
      )
    )
  )
);

```

```

        FROM movimentacoes_estoque m
        WHERE m.produto_id = p.id
    ),
    0
)
);

```

```

INSERT INTO configuracoes_backup (
    banco,
    caminho_destino,
    qtd_manter,
    caminho_copia,
    criptografar,
    compactar
)
VALUES (
    'aquarismo',
    '/var/backups/aquarismo',
    7,
    '/var/backups/aquarismo/copia',
    false,
    true
);

```

```

DO $$
DECLARE
    total_registros BIGINT;
BEGIN

```

```

    SELECT
        (SELECT COUNT(*) FROM especies)
        + (SELECT COUNT(*) FROM clientes)
        + (SELECT COUNT(*) FROM produtos_acessorios)
        + (SELECT COUNT(*) FROM lotes)
        + (SELECT COUNT(*) FROM pedidos)
        + (SELECT COUNT(*) FROM itens_pedido)
        + (SELECT COUNT(*) FROM movimentacoes_estoque)
        + (SELECT COUNT(*) FROM configuracoes_backup)
        + (SELECT COUNT(*) FROM execucoes)
        + (SELECT COUNT(*) FROM logs_execucao)

```

```

    INTO total_registros;

```

1.4 DELETE ALEATÓRIO

```
DELETE FROM itens_pedido
WHERE id IN (
    SELECT id
    FROM itens_pedido
    ORDER BY RANDOM()
    LIMIT (
        SELECT FLOOR(COUNT(*) * 0.05)::INTEGER
        FROM itens_pedido
    )
);
```

```
DELETE FROM pedidos
WHERE id IN (
    SELECT id
    FROM pedidos
    ORDER BY RANDOM()
    LIMIT (
        SELECT FLOOR(COUNT(*) * 0.05)::INTEGER
        FROM pedidos
    )
);
```

```
DELETE FROM movimentacoes_estoque
WHERE id IN (
    SELECT id
    FROM movimentacoes_estoque
    ORDER BY RANDOM()
    LIMIT (
        SELECT FLOOR(COUNT(*) * 0.08)::INTEGER
```

```
        FROM movimentacoes_estoque
    )
);
```

```
DELETE FROM clientes
WHERE id IN (
    SELECT c.id
    FROM clientes c
    WHERE NOT EXISTS (
        SELECT 1
        FROM pedidos p
        WHERE p.cliente_id = c.id
    )
    ORDER BY RANDOM()
    LIMIT (
        SELECT FLOOR(COUNT(*) * 0.10)::INTEGER
        FROM clientes c
        WHERE NOT EXISTS (
            SELECT 1
            FROM pedidos p
            WHERE p.cliente_id = c.id
        )
    )
);
```

```
DELETE FROM produtos_acessorios
WHERE id IN (
    SELECT p.id
    FROM produtos_acessorios p
    WHERE NOT EXISTS (
```

```

SELECT 1
FROM movimentacoes_estoque m
WHERE m.produto_id = p.id
)
AND NOT EXISTS (
    SELECT 1
    FROM itens_pedido i
    WHERE i.produto_id = p.id
)
ORDER BY RANDOM()
LIMIT (
    SELECT FLOOR(COUNT(*) * 0.10)::INTEGER
    FROM produtos_acessorios p
    WHERE NOT EXISTS (
        SELECT 1
        FROM movimentacoes_estoque m
        WHERE m.produto_id = p.id
    )
    AND NOT EXISTS (
        SELECT 1
        FROM itens_pedido i
        WHERE i.produto_id = p.id
    )
)
);

```

```

DELETE FROM lotes
WHERE id IN (
    SELECT l.id
    FROM lotes l

```

```

WHERE NOT EXISTS (
    SELECT 1
    FROM movimentacoes_estoque m
    WHERE m.lote_id = l.id
)
AND NOT EXISTS (
    SELECT 1
    FROM itens_pedido i
    WHERE i.lote_id = l.id
)
ORDER BY RANDOM()
LIMIT (
    SELECT FLOOR(COUNT(*) * 0.10)::INTEGER
    FROM lotes l
    WHERE NOT EXISTS (
        SELECT 1
        FROM movimentacoes_estoque m
        WHERE m.lote_id = l.id
    )
    AND NOT EXISTS (
        SELECT 1
        FROM itens_pedido i
        WHERE i.lote_id = l.id
    )
)
);

```

```

DELETE FROM especies
WHERE id IN (
    SELECT e.id

```

```
FROM especies e
WHERE NOT EXISTS (
    SELECT 1
    FROM lotes l
    WHERE l.especie_id = e.id
)
ORDER BY RANDOM()
LIMIT (
    SELECT FLOOR(COUNT(*) * 0.10)::INTEGER
    FROM especies e
    WHERE NOT EXISTS (
        SELECT 1
        FROM lotes l
        WHERE l.especie_id = e.id
    )
)
);
```

```
UPDATE lotes l
SET quantidade_atual = GREATEST(
    0,
    l.quantidade_inicial
    +
    COALESCE(
        (
            SELECT SUM(
                CASE
                    WHEN m.tipo = 'entrada' THEN m.quantidade
                    WHEN m.tipo IN ('venda', 'perda')
                    THEN -m.quantidade
```

```

        ELSE 0
    END
)
FROM movimentacoes_estoque m
WHERE m.lote_id = l.id
),
0
)
);
UPDATE produtos_acessorios p
SET estoque_atual = GREATEST(
    0,
    1000
    +
    COALESCE(
        (
            SELECT SUM(
                CASE
                    WHEN m.tipo = 'entrada' THEN m.quantidade
                    WHEN m.tipo IN ('venda', 'perda')
                        THEN -m.quantidade
                    ELSE 0
                END
            )
            FROM movimentacoes_estoque m
            WHERE m.produto_id = p.id
        ),
        0
    )
);

```

```

DO $$
DECLARE
    total_registros BIGINT;
BEGIN
    SELECT
        (SELECT COUNT(*) FROM especies)
        + (SELECT COUNT(*) FROM clientes)
        + (SELECT COUNT(*) FROM produtos_acessorios)
        + (SELECT COUNT(*) FROM lotes)
        + (SELECT COUNT(*) FROM pedidos)
        + (SELECT COUNT(*) FROM itens_pedido)
        + (SELECT COUNT(*) FROM movimentacoes_estoque)
        + (SELECT COUNT(*) FROM configuracoes_backup)
        + (SELECT COUNT(*) FROM execucoes)
        + (SELECT COUNT(*) FROM logs_execucao)
    INTO total_registros;

    RAISE NOTICE '=====';
    RAISE NOTICE 'TOTAL APÓS REMOÇÕES: %', total_registros;
    RAISE NOTICE '=====';
END $$;

```

2. EVIDÊNCIA DO BANCO POPULADO

Object Explorer

▼ SDBC

▼ Servers

▼ PostgreSQL 18

▼ Databases (3)

▼ SDBC

▼ Casts

▼ Catalogs

▼ Event Triggers

▼ Extensions

▼ Foreign Data Wrappers

▼ Languages

▼ Publications

▼ Schemas (1)

▼ Subscriptions

▼ dadosII

▼ postgres

▼ Login/Group Roles

▼ Tablespaces

SDBC/postgres@PostgreSQL 18*

SDBC/postgres@PostgreSQL 18

Query Query History

83

84

85

86

87

SELECT COUNT(*) FROM lotes

)

+ (

SELECT COUNT(*) FROM pedidos

)

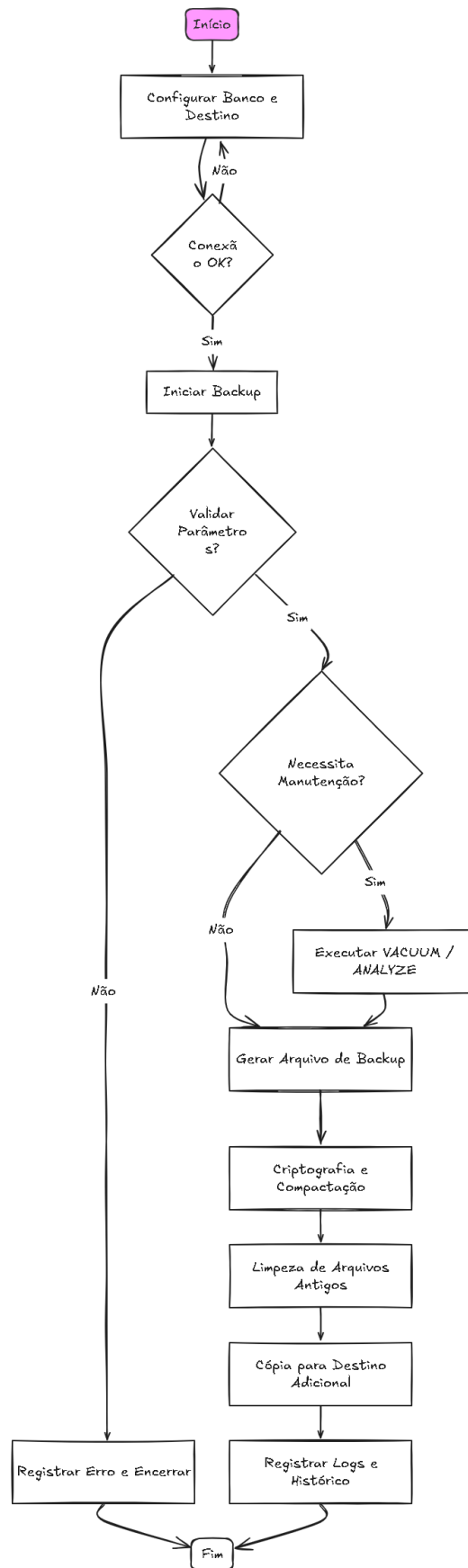
Data Output Messages Notifications

SQL

Showing rows: 1 t

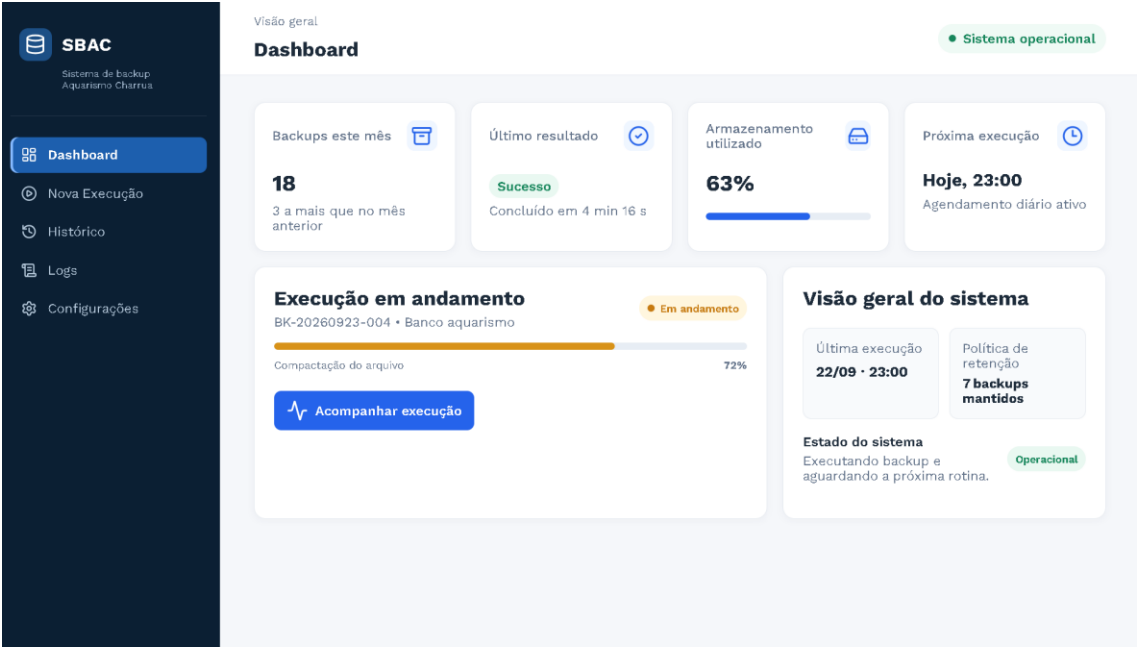
	tabela	quantidade
	text	bigint
1	TOTAL GERAL	463024
2	execucooes	0
3	configuracoes_backup	1
4	logs_execucao	0
5	especies	15
6	produtos_acessorios	2000
7	lotes	10000
8	clientes	29995
9	pedidos	66500
10	itens_pedido	126367
11	movimentacoes_estoq...	228146

3. DIAGRAMA DE FUNCIONAMENTO

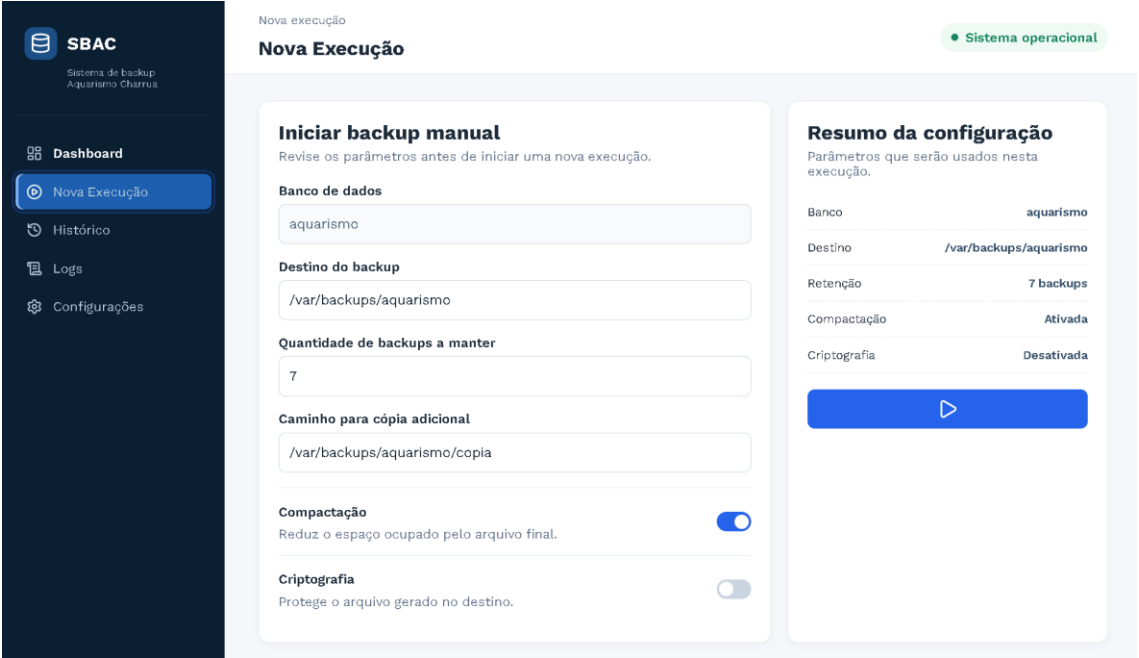


4. PROTÓTIPO DAS TELAS

4.1 DASHBOARD



4.2 NOVA EXECUÇÃO



4.3 HISTÓRICO

SBAC

Sistema de backup

Aquarismo Charrua

Dashboard

Nova Execução

Histórico

Logs

Configurações

Histórico

Sistema operacional

Histórico de execuções

Consulte resultados, decisões de manutenção e regras aplicadas.

Status

Todos

Período

Últimos 30 dias

Banco

aquarismo

Aplicar filtros

ID	INÍCIO	TÉRMINO	STATUS	DECISÃO DE MANUTENÇÃO	REGRA APLICADA	RESULTADO	AÇÃO
BK-20260923-004	23/09/2026 23:00	—	Em andamento	Em avaliação	Manter últimos 7	Processando compactação	Detalhes
BK-20260922-003	22/09/2026 23:00	22/09/2026 23:04	Sucesso	Mantido	Manter últimos 7	Arquivo validado (1,8 GB)	Detalhes
BK-20260921-002	21/09/2026 23:00	21/09/2026 23:02	Falha	Não aplicada	Manter últimos 7	Espaço temporário insuficiente	Detalhes
BK-20260920-001	20/09/2026 23:00	20/09/2026 23:05	Sucesso	Removido backup antigo	Manter últimos 7	Arquivo validado (1,8 GB)	Detalhes

4.4 LOGS

SBAC

Sistema de backup

Aquarismo Charrua

Dashboard

Nova Execução

Histórico

Logs

Configurações

Logs

Sistema operacional

Logs técnicos

Registros operacionais das execuções realizadas pelo SBAC.

Execução

Todas as execuções

Etapas

Todas as etapas

Filtrar logs

EXECUÇÃO	ETAPA	MENSAGEM	DATA/HORA	SAÍDA TÉCNICA
BK-20260923-004	Compactação	Compactando dump SQL	23/09 23:03:22	gzip: processando aquarismo.sql
BK-20260922-003	Verificação	Integridade do arquivo confirmada	22/09 23:04:15	sha256 verified: 4bd9...8ac2
BK-20260921-002	Criação	Falha ao reservar espaço temporário	21/09 23:01:49	ERROR: No space left on device

4.5 CONFIGURAÇÕES

 SBAC

Sistema de backup
Aqurismo Charrua

Dashboard

Nova Execução

Histórico

Logs

Configurações

Configurações

Configurações

Configurações de backup

Esta configuração define como os novos backups serão executados.

Banco

aquarismo

Caminho de destino

/var/backups/aqurismo

Quantidade de backups

7

Caminho da cópia adicional

/var/backups/aqurismo/copia

Criptografar

☐

Compactar

☒

Salvar

Configuração ativa

Os parâmetros salvos serão aplicados à próxima execução manual ou agendada. A política de retenção remove apenas backups que ultrapassarem o limite definido.